

setup-inicial

Configuración inicial: autenticación con API key y secret

Esta guía explica cómo dejar el **repo esqueleto** listo para llamar endpoints protegidos con cabeceras `x-api-key` y `x-api-secret`. Es el primer paso práctico antes del laboratorio de Textract de la clase 1.

Si no has usado NestJS o Node, lee primero la [visión global](#) y luego sigue los pasos en orden.

Qué ya viene en el esqueleto y qué añades tú

Ya incluido en el repo (no lo reescribas)	Lo implementas en esta guía
<code>package.json</code> con dependencias y scripts	Archivos bajo <code>src/auth/</code>
<code>src/database/data-source.ts</code>	<code>src/modulo1/clase01/clase01.controller.ts</code>
<code>src/database/database.module.ts</code>	Cambios en <code>src/modulo1/modulo1.module.ts</code>
<code>src/entities/api-client.entity.ts</code>	
<code>src/migrations/...CreateApiClients.ts</code>	
<code>scripts/seed-test-api-client.ts</code>	
<code>AppModule</code> con <code>ConfigModule</code> y <code>DatabaseModule</code>	

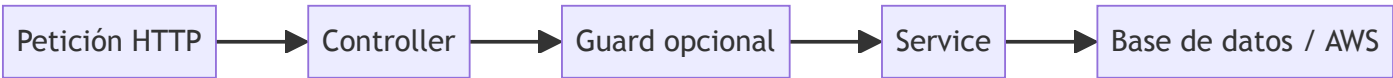
Al terminar podrás probar:

```
curl -H "x-api-key: test1" -H "x-api-secret: pass1" \
http://localhost:3000/modulo1/clase01/test
```

Respuesta esperada: texto endpoint test autenticado.

Visión global (sin asumir que ya conoces NestJS)

Node.js ejecuta JavaScript/TypeScript en el servidor. **NestJS** es un marco que organiza la API en piezas con nombres fijos:



Pieza	Rol en una frase
Module	Agrupar controllers, services y guards que van juntos
Controller	Define rutas (GET /modulo1/clase01/test)
Guard	Se ejecuta <i>antes</i> del controller; aquí valida API key/secret
Service	Lógica de negocio (Textract, Glue, etc.; en este paso aún no lo usamos)
Entity	Clase que describe una tabla de Postgres (ApiClient); todas viven en src/entities/

TypeORM conecta esas entidades con Postgres. Las **migraciones** crean o cambian tablas; el **seed** inserta datos de prueba.

Orden recomendado de trabajo

1. `npm install` y archivo `.env`
2. `npm run migration:run`
3. `npm run seed:api-client`
4. Crear `AuthModule`, `ApiKeyGuard`, controller y registrar en `Modulo1Module`
5. `npm run start:dev` y probar con `curl`

1. package.json — scripts y dependencias

El esqueleto ya trae un `package.json` alineado con la solución del formador. Solo necesitas entender qué hace cada script:

Script	Comando real	Para qué sirve
<code>build</code>	<code>nest build</code>	Compila TypeScript a <code>dist/</code> (producción, Render)
<code>start</code>	<code>nest start</code>	Arranca la API una vez (sin recargar al guardar)
<code>start:dev</code>	<code>nest start --watch</code>	Desarrollo: reinicia al cambiar archivos
<code>start:prod</code>	<code>node dist/main</code>	Producción tras <code>npm run build</code>
<code>migration:run</code>	TypeORM CLI	Aplica migraciones pendientes en Postgres
<code>migration:revert</code>	TypeORM CLI	Deshace la última migración
<code>migration:show</code>	TypeORM CLI	Lista qué migraciones están aplicadas
<code>seed:api-client</code>	<code>ts-node scripts/seed-test-api-client.ts</code>	Inserta un cliente API de prueba en la BD

Dependencias relevantes para la autenticación:

Paquete	Uso en este paso
<code>@nestjs/config</code>	Lee variables del <code>.env</code>
<code>@nestjs/typeorm + typeorm + pg</code>	Conexión y consultas a Postgres
<code>bcrypt</code>	Hashea el <code>secret</code> en el <code>seed</code> ; el <code>guard</code> compara sin guardar el <code>secret</code> en claro

No hace falta instalar nada extra si ya hiciste `npm install` en la raíz del esqueleto.

2. AppModule — qué hay en esqueleto vs solución

Archivo: src/app.module.ts

En el **esqueleto actual** y en la **solución** el AppModule es el mismo para esta parte:

```
import { Module } from '@nestjs/common';
import { ConfigModule } from '@nestjs/config';
import { DatabaseModule } from '../database/database.module';
import { HealthModule } from '../health/health.module';
import { Modulo1Module } from '../modulo1/modulo1.module';
import { Modulo2Module } from '../modulo2/modulo2.module';

@Module({
  imports: [
    ConfigModule.forRoot({ isGlobal: true }),
    DatabaseModule,
    HealthModule,
    Modulo1Module,
    Modulo2Module,
  ],
})
export class AppModule {}
```

Para qué sirve cada import

Import	Función
ConfigModule.forRoot({ isGlobal: true })	Carga .env y permite usar ConfigService en cualquier módulo sin volver a importar ConfigModule
DatabaseModule	Abre la conexión global a Postgres (TypeOrmModule.forRootAsync) usando DATABASE_URL y DATABASE_SCHEMA
HealthModule	Endpoint público GET /health (Render y comprobaciones)

Import	Función
Modulo1Module	Agrupar los controladores del módulo 1 (clase 01, 02, ...)
Modulo2Module	Reservado para la fase n8n + LLM

Diferencia importante con versiones anteriores del material: si ves ejemplos con DatabaseService y pg directo, no los uses en este esqueleto. En el curso la base de datos va con **TypeORM** (database.module.ts + entidades + migraciones). Para autenticación y para el resto del módulo 1 usa siempre este enfoque.

No registres AuthModule en AppModule. Se importa solo desde Modulo1Module, así cada módulo de clase controla sus propios guards.

3. AuthModule — cómo crearlo y para qué sirve

Crea la carpeta src/auth/ y el archivo src/auth/auth.module.ts:

```
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { ApiClient } from '../entities/api-client.entity';
import { ApiKeyGuard } from '../guards/api-key.guard';

@Module({
  imports: [TypeOrmModule.forFeature([ApiClient])],
  providers: [ApiKeyGuard],
  exports: [ApiKeyGuard, TypeOrmModule],
})
export class AuthModule {}
```

Línea a línea (idea general)

Código	Para qué
TypeOrmModule.forFeature([ApiClient])	Registra el repositorio de la tabla api_clients en este módulo para poder inyectarlo en el guard

Código	Para qué
providers: [ApiKeyGuard]	Nest crea una instancia del guard cuando haga falta
exports: [ApiKeyGuard, TypeOrmModule]	El guard y el repositorio ApiClient quedan disponibles en Modulo1Module cuando usas @UseGuards(ApiKeyGuard)

Sin forFeature, el guard no podría hacer findOne en Postgres. Sin exportar TypeOrmModule, Nest suele fallar con UnknownDependenciesException (ApiClientRepository no encontrado en Modulo1Module).

4. ApiKeyGuard — cómo crearlo y para qué sirve

Crea src/auth/guards/api-key.guard.ts:

```
import {
  CanActivate,
  ExecutionContext,
  Injectable,
  UnauthorizedException,
} from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import * as bcrypt from 'bcrypt';
import { Request } from 'express';
import { Repository } from 'typeorm';
import { ApiClient } from '../../entities/api-client.entity';

export type AuthenticatedRequest = Request & { apiClient: ApiClient };

@Injectable()
export class ApiKeyGuard implements CanActivate {
  constructor(
    @InjectRepository(ApiClient)
    private readonly apiClientsRepository: Repository<ApiClient>,
  ) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
```

```
const request = context.switchToHttp().getRequest<Request>();
const apiKey = this.readHeader(request, 'x-api-key');
const apiSecret = this.readHeader(request, 'x-api-secret');

if (!apiKey || !apiSecret) {
  throw new UnauthorizedException('Missing x-api-key or x-api-secret headers');
}

const client = await this.apiClientsRepository.findOne({
  where: { apiKey, isActive: true },
});

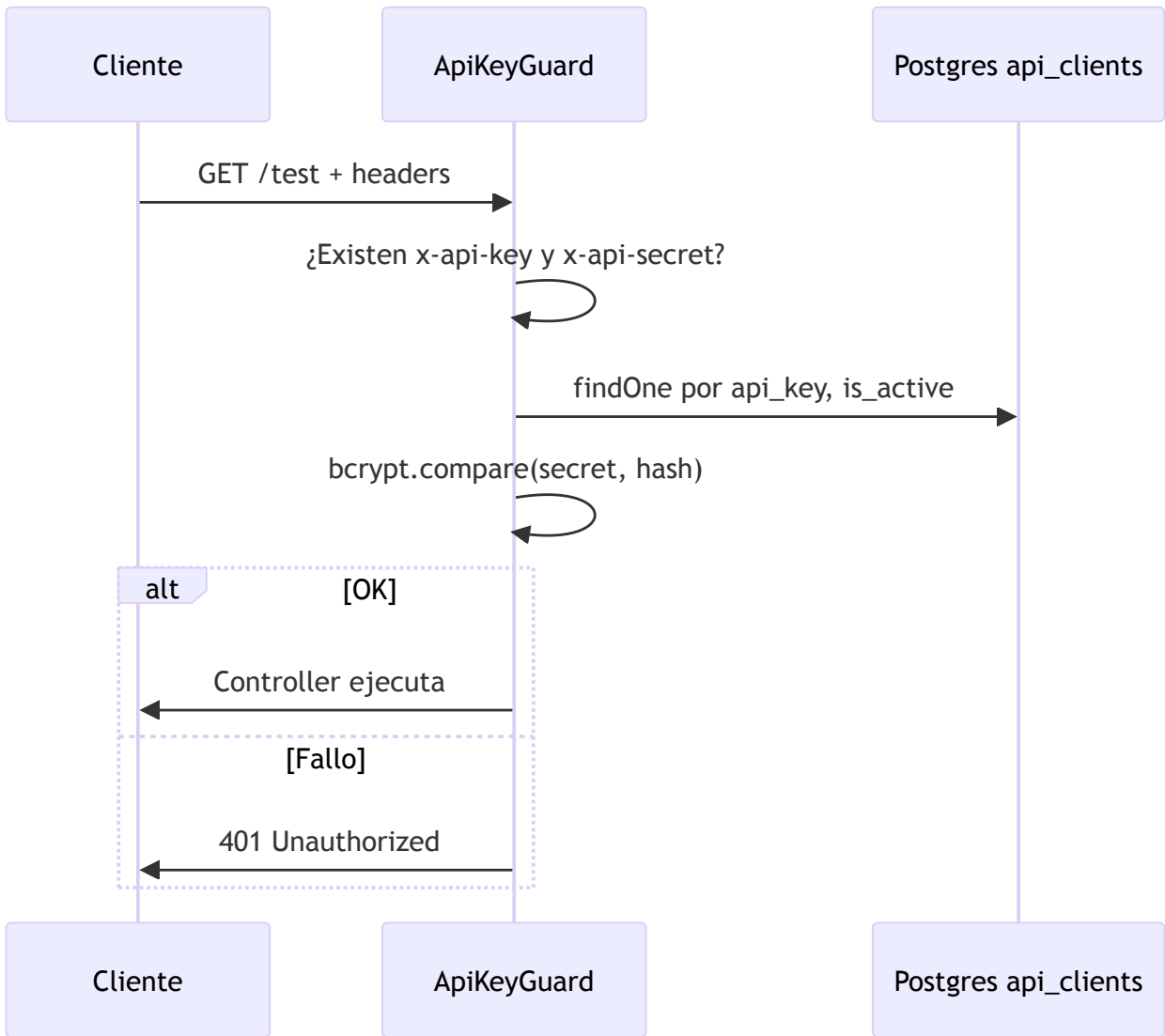
if (!client) {
  throw new UnauthorizedException('Invalid API credentials');
}

const secretMatches = await bcrypt.compare(apiSecret,
  client.apiSecretHash);
if (!secretMatches) {
  throw new UnauthorizedException('Invalid API credentials');
}

(request as AuthenticatedRequest).apiClient = client;
return true;
}

private readHeader(request: Request, name: string): string | undefined {
  const value = request.headers[name];
  if (Array.isArray(value)) {
    return value[0];
  }
  return value;
}
}
```

Flujo del guard



Concepto	Explicación breve
CanActivate	Contrato de Nest: método canActivate devuelve true o lanza error
ExecutionContext	Acceso a la petición HTTP actual
@InjectRepository(ApiClient)	TypeORM inyecta el repositorio de la entidad
bcrypt.compare	Compara el secret en claro de la cabecera con el hash guardado en BD (nunca guardamos el secret en texto plano)

Concepto	Explicación breve
AuthenticatedRequest	Tras validar, adjunta apiClient al request por si el controller lo necesita después

Seguridad: si falta cabecera, key incorrecta o secret incorrecto → 401 con mensaje en inglés (convención del código del curso).

5. Modulo1Module — registrar auth y el controller

El esqueleto trae Modulo1Module vacío. Sustituye el contenido de src/modulo1/modulo1.module.ts por:

```
import { Module } from '@nestjs/common';
import { AuthModule } from '../auth/auth.module';
import { Clase01Controller } from './clase01/clase01.controller';

@Module({
  imports: [AuthModule],
  controllers: [Clase01Controller],
})
export class Modulo1Module {}
```

Parte	Para qué
imports: [AuthModule]	Trae ApiKeyGuard exportado y el repositorio de ApiClient
controllers: [Clase01Controller]	Publica las rutas bajo /modulo1/clase01/...

En clases posteriores añadirás más controllers al array controllers y, si hace falta, más providers (services de Textract, etc.).

6. Clase01Controller — endpoint de prueba autenticado

Crea src/modulo1/clase01/clase01.controller.ts:

```
import { Controller, Get, UseGuards } from '@nestjs/common';
import { ApiKeyGuard } from '../../auth/guards/api-key.guard';

@Controller('modulo1/clase01')
export class Clase01Controller {
  @Get('test')
  @UseGuards(ApiKeyGuard)
  testAuthenticated(): string {
    return 'endpoint test autenticado';
  }
}
```

Decorador / código	Para qué
@Controller('modulo1/clase01')	Prefijo de ruta: todas las rutas de esta clase empiezan así
@Get('test')	Ruta completa: GET /modulo1/clase01/test
@UseGuards(ApiKeyGuard)	Solo esta ruta exige autenticación; /health sigue siendo pública
Respuesta string	Nest envía el texto como cuerpo de la respuesta

Más adelante en la misma clase añadirás POST /modulo1/clase01/texttract/text **sin** quitar el guard en las rutas que deban estar protegidas (o protegerás todo el controller según indique el formador).

7. Variables de entorno (.env)

Copia el ejemplo y rellena valores reales (nunca subas .env a Git):

```
cp .env.example .env
```

Referencia (esqueleto/.env.example):

```
PORT=3000
```

```
# AWS – región Virginia; bucket y claves de tu usuario IAM del curso
```

```
AWS_REGION=us-east-1
```

```
AWS_ACCESS_KEY_ID=
```

```
AWS_SECRET_ACCESS_KEY=
```

```
AWS_S3_BUCKET=grupo1-980921750553-us-east-1-an
```

```
# Postgres (Supabase): URL del pooler y esquema de tu grupo/docente
```

```
DATABASE_URL=postgresql://usuario:password@host:6543/postgres?sslmode=require
```

```
DATABASE_SCHEMA=grupo1
```

```
# Valores que usará el script de seed (opcional; si no están, usa test-api-key  
/ test-api-secret)
```

```
SEED_API_KEY=test1
```

```
SEED_API_SECRET=pass1
```

Variable	Obligatoria para auth	Descripción
PORT	No (default 3000)	Puerto HTTP local
AWS_*	Para Textract, no para /test	Las usarás en la misma clase al conectar S3/Textract
DATABASE_URL	Sí	Cadena de conexión Postgres (usuario/contraseña del curso)
DATABASE_SCHEMA	Sí	Esquema donde vive tu tabla (docente, grupo1, ...). Debe existir en Supabase antes de migrar
SEED_API_KEY	No	API key que insertará el seed
SEED_API_SECRET	No	Secret en claro; el script guarda solo su hash

Cabeceras HTTP en las peticiones (no van en `.env` del servidor; las envía el cliente):

- `x-api-key`: debe coincidir con `api_key` en la tabla (p. ej. `test1` si usaste ese seed).
 - `x-api-secret`: el secret en claro (p. ej. `pass1`); el servidor lo compara con el hash.
-

8. Migraciones — cómo correrlas y cómo funcionan

Qué es una migración

Un archivo en `src/migrations/` describe un cambio de esquema (crear tabla, añadir columna...). TypeORM guarda en Postgres qué migraciones ya se ejecutaron para no repetir las.

La migración del curso crea la tabla `api_clients` en **tu esquema** (`DATABASE_SCHEMA`):

- `id` (UUID)
- `name`, `api_key` (único), `api_secret_hash`, `is_active`, `created_at`

El esqueleto **no** crea el esquema (`CREATE SCHEMA`): en Supabase el formador ya creó el esquema de tu grupo.

Comandos

Con `.env` cargado (el CLI lee `data-source.ts`, que hace `dotenv.config()`):

```
# Ver estado
npm run migration:show

# Aplicar migraciones pendientes
npm run migration:run

# Deshacer solo la última (cuidado en datos compartidos)
npm run migration:revert
```

Cómo se conecta el CLI

`src/database/data-source.ts` exporta un `DataSource` de TypeORM con:

- URL y SSL ajustados (`resolvePostgresConnection` — necesario en Supabase)
- `entities: ApiClient`

- migrations: src/migrations/*.ts

Eso es **independiente** de DatabaseModule: el CLI no arranca Nest; solo ejecuta SQL vía TypeORM.

Errores frecuentes

Síntoma	Qué revisar
relation "api_clients" does not exist	No corriste migration:run o DATABASE_SCHEMA incorrecto
Error SSL / timeout	DATABASE_URL del pooler, región, contraseña
permission denied for schema	Esquema inexistente o usuario sin permiso en Supabase

9. Seed — seed-test-api-client y cómo correrlo

Para qué sirve

Inserta **una fila de prueba** en api_clients con:

- api_key = valor de SEED_API_KEY (o test-api-key por defecto)
- api_secret_hash = bcrypt del SEED_API_SECRET (o test-api-secret por defecto)

El secret **no** se guarda en claro en la base de datos.

Cómo ejecutarlo

```
npm run seed:api-client
```

Con valores explícitos sin tocar .env:

```
SEED_API_KEY=test1 SEED_API_SECRET=pass1 npm run seed:api-client
```

Salida esperada si todo va bien:

```
Created API client:  
  x-api-key: test1  
  x-api-secret: pass1
```

Si la key ya existe:

```
API client already exists for key: test1
```

En ese caso usa otra key, borra la fila en Supabase (con cuidado) o sigue usando la existente.

Qué hace el script por dentro

1. Lee `.env` (`DATABASE_URL`, `DATABASE_SCHEMA`, `SEED_*`)
2. Crea un `DataSource` temporal (misma lógica SSL que migraciones)
3. Busca si ya hay fila con esa `api_key`
4. Si no hay, `bcrypt.hash(secret, 10)` y save en `api_clients`
5. Cierra la conexión

No necesitas arrancar Nest (`start:dev`) para el seed.

Verificación final

```
npm install  
cp .env.example .env    # editar DATABASE_* y SEED_*  
  
npm run migration:run  
npm run seed:api-client  
  
npm run start:dev
```

En otra terminal:

```
# Público  
curl http://localhost:3000/health  
  
# Protegido – sustituye test1/pass1 por tus valores de seed  
curl -H "x-api-key: test1" -H "x-api-secret: pass1" \
```

```
http://localhost:3000/modulo1/clase01/test
```

```
# Sin cabeceras → 401
```

```
curl -i http://localhost:3000/modulo1/clase01/test
```

Checklist

- ☐ .env con DATABASE_URL y DATABASE_SCHEMA correctos
 - ☐ npm run migration:run sin errores
 - ☐ npm run seed:api-client creó o reconoce el cliente
 - ☐ Existen AuthModule, ApiKeyGuard, Clase01Controller
 - ☐ Modulo1Module importa AuthModule y declara Clase01Controller
 - ☐ GET /modulo1/clase01/test responde 200 con cabeceras correctas
 - ☐ La misma ruta responde 401 sin cabeceras o con credenciales malas
-

Siguiente paso

Cuando la autenticación funcione, continúa con la [parte práctica de Textract](#) de la clase 1 (POST /modulo1/clase01/textract/text). Los nuevos endpoints de documentos también deberán estar protegidos con el mismo patrón (@UseGuards(ApiKeyGuard) o a nivel de controller).